algorithm to it (or to the example in figure 14.9, exactly how we did for BFS), and reconstruct the shortest path.

The result for this section's example is shown in figure 14.18, where we computed and showed the shortest distance from the source vertex (v_S) to every other vertex; notice that there are some edges, drawn with a thin dotted line, that doesn't belong to any shortest path, while the shortest path to our destination, v_G, is highlighted with thick dashed lines.

This example is perfect to illustrate the need of such algorithms as Dijkstra's, because there are two paths between v_S and v_G that sum up almost to the same distance, and our intuition would likely go for the longest one, because it looks more linear.

One final consideration: as we mentioned, applying the algorithm was straightforward, but only because of one property of this graph: it doesn't have any edge with negative weight. Any such edge, in fact, would violate the assumption behind the algorithm, i.e. that if we expand the frontier of visited vertices by choosing the closest unvisited vertex at each iteration, then at the time a vertex is visited, we know its minimum distance from start.

This happens because Dijkstra's algorithm (like BFS) is a *greedy* algorithm, a kind of algorithm that can find the solution to a problem by making locally-optimal choices: in fact, to decide which vertex to visit next, we only need to take into account the outgoing edges of the vertices we already visited. Greedy algorithms can only be applied to certain problems: having negative edges makes the problem